On the Limitations of Perturbation-Based Methods for Training Deep Networks

Yash Mehta[1], **Naoki Hiratani**[1], **Peter Humphreys**[2], **Peter E. Latham**[1], **Timothy Lillicrap**[2]
[1]Gatsby Computational Neuroscience Unit, University College London
[2]DeepMind, London, UK

Abstract

What algorithms drive goal directed learning in networks in the brain? In machine learning, networks are almost exclusively trained using stochastic gradient descent, which delivers errors tailored to each neuron in a network. However, computing such tailored errors requires complicated machinery that is unlikely to exist in the brain. An alternative is to apply random perturbations to the weights, see whether that increases or decreases a global error, and then adjust the weights accordingly. The fact that this approach does not require complicated machinery, along with evidence that the brain utilizes global signals for learning, has prompted neuroscientists to speculate that the brain may use perturbation algorithms. However, little is known about the efficacy of these algorithms for training large networks to perform complex tasks. Here we address this issue, with a focus on node perturbation (a fast version of weight perturbation). We start by reviewing why node perturbation is slower than stochastic gradient descent in convex systems. We then investigate empirically how it scales on large and convolutional network architectures, and on increasingly difficult image classification tasks. As expected, we find that node perturbation is slower than stochastic gradient descent. For small networks trained on simple datasets, it is only slightly slower; for large networks trained on complex datasets, it is at least hundreds of times slower, making it impractical as a learning algorithm.

1 Introduction

The brain learns by modifying synaptic connections between neurons [1, 2]. Determining how to change the approximately 10^{14} synapses (in humans) to improve goal-oriented behaviours is, however, a hard problem. In deep artificial neural networks, this is solved by computing gradients via backpropagation of error. The resulting algorithm is known as backprop [3, 4], and has revolutionized machine learning: deep networks now match, and sometimes exceed, human performance on a variety of tasks, including speech [5], image recognition [6, 7], language translation [8], game playing [9, 10, 11], chess [12], and robotics [13, 14]. However, backprop has a downside: it's complicated. It computes the gradient of a global objective with respect to each synaptic weight by recursively computing gradients layer by layer, a procedure that requires information to be transmitted from higher to lower layers. This computation requires complex circuitry that the brain is unlikely to possess [15, 16, 17]; it is, therefore, unlikely that backprop is used in the brain.

Recent work has posited that the brain uses approximations to the brackprop algorithm that are more biologically plausible than the vanilla version used in deep networks [18, 19, 20, 21, 22]. However, there is a simpler class of learning algorithms — algorithms that do not require complex machinery to compute gradients, and, importantly, align well with the established neurophysiology of the brain's learning systems [23]. Instead of explicitly computing gradients, these algorithms add small random perturbations to each weight and ask how that affects performance. If performance

Preprint. Under review.

improves, the weight perturbations are kept; if performance gets worse, weights are moved in the opposite direction of the perturbation. Alternatively, perturbations can be added at the single neuron level rather than the single synapse level, which reduces variability; this results in classic algorithms such as REINFORCE [24] and node perturbation [25, 26]. These algorithms require only a source of noise and a global signal conveying information about performance. Both are consistent with known neurophysiology: synapses [27] and networks [28] are instrinsically noisy, and global neuromodulatory signals that strongly modulate synaptic plasticity with high temporal precision [29] are known to project everywhere in the brain [30]. Moreover, these algorithms are applicable to a range of architectures [31], including recurrent networks [32], and they have been proposed to underly the learning mechanism of diverse neural systems, including song generation circuits [33, 34], cerebellum [35], and pre-motor systems [32, 36].

There is, though, a downside to perturbation-based approaches: estimating a gradient by taking random steps is a slow process, especially in high dimensions (the common case for neural networks). Thus, these methods tend to be slower than gradient-based ones. For that reason, modern deep learning relies on gradient-based learning almost exclusively. This does not rule out perturbation-based methods in the brain, which may opt for simplicity over speed.

To help understand when node perturbation is a viable option in the brain, it is necessary to know how much slower it is than stochastic gradient descent. Werfel and colleagues [26] studied perturbation algorithms thoroughly in a shallow linear regression setting, but they did not investigate more complex scenarios, such as deep nonlinear networks. Payeur and colleagues [37] applied node perturbation to image classification tasks, with results suggesting that node perturbation is significantly slower than SGD. However, we are not aware of any systematic study to delineate the limitations of perturbation methods for training large networks. Here, we make the following contributions:

1. We review theory that highlights the relationship between node perturbation and SGD in deep networks.
2. We develop a straightforward approach to leverage auto-differentiation functionality of modern deep learning libraries to run node perturbation with convolutional network architectures. This enables like-for-like comparisons with SGD experiments.
3. We show that the empirical results for deep networks follow that predicted by theory in the case of small learning rates.
4. Where theory is intractable, we carry out a rigorous empirical study of node perturbation for training deep networks to perform complex image classification tasks.
5. We find empirically that for large networks trained on complex tasks, node perturbation is orders of magnitude slower than SGD, and far more likely to lead to 'crashes', where we see a sudden fall in the performance of the network.

2 Background

2.1 The node perturbation update rule

Our focus here is on feedforward networks, of the form

$$\mathbf{x}^l = \phi(\mathbf{h}^{l-1}) \tag{1a}$$

$$\mathbf{h}^l = \mathbf{w}^l \cdot \mathbf{x}^l \tag{1b}$$

where l, which ranges from 1 to L, refers to layer, and ϕ is a pointwise nonlinearity. The output of the network, $\hat{\mathbf{y}}$, is a function of $\mathbf{x}^L$. The input is $\mathbf{x}^0$, which we'll refer to as $\mathbf{x}$, and we'll use $\mathbf{y}(\mathbf{x})$ to denote the desired output given the input, $\mathbf{x}$. The resulting loss, for a particular $\mathbf{x}, \mathbf{y}$ pair, is denoted $\mathcal{L}(\mathbf{x}, \mathbf{y}(\mathbf{x}), \hat{\mathbf{y}}; \mathbf{w})$, which we'll generally abbreviate as $\mathcal{L}(\mathbf{x}; \mathbf{w})$.

An especially efficient way to update the weights is via stochastic gradient descent (SGD),

$$\Delta \mathbf{w}_{\text{sgd}} = -\eta \, \frac{\partial \mathcal{L}(\mathbf{x}; \mathbf{w})}{\partial \mathbf{w}} \, , \tag{2}$$

and an especially efficient way to compute the derivative on the right hand side is via backprop [3]. However, that's not the only way. A classic, if noisy, approach, known as weight perturbation [38],

is to perturb the weights by a small amount and monitor change in loss. In particular, consider the update rule

$$\Delta \mathbf{w}_{\text{wp}} = -\eta \frac{\mathcal{L}(\mathbf{x}; \mathbf{w} + \sigma \boldsymbol{\xi}_{\mathbf{w}}) - \mathcal{L}(\mathbf{x}; \mathbf{w})}{\sigma} \boldsymbol{\xi}_{\mathbf{w}} \tag{3}$$

where $\boldsymbol{\xi}_{\mathbf{w}}$ is an uncorrelated, zero mean, unit variance Gaussian random variable and $\sigma \ll 1$. Taylor expanding the loss to first order in σ and averaging over the noise, we see that $\Delta \mathbf{w}_{\text{wp}}$ is equal to $\Delta \mathbf{w}_{\text{sgd}}$ on average.

The problem with this approach, of course, is that it has extremely high variance. That variance can, however, be reduced. Examining Eq. (1b), we see that a very large manifold of weight perturbations leads to the same perturbation in $\mathbf{h}^l$: if we let $\mathbf{w}_l \to \mathbf{w}_l + \sigma \boldsymbol{\xi}^l_{\mathbf{w}}$, then $\mathbf{h}^l \to \mathbf{h}^l + \sigma \boldsymbol{\xi}^l_{\mathbf{w}} \cdot \mathbf{x}^l$. It makes far more sense, therefore, to perturb the "nodes", $\mathbf{h}^l$, rather than the weights, $\mathbf{w}^l$ – an algorithm known as node perturbation [25, 26].

The node perturbation update rule is essentially the same as the one for weight perturbation, Eq. (3), except with weights replaced by nodes. To do that in practice, we add noise to Eq. (1b),

$$\mathbf{h}^l = \mathbf{w}^l \cdot \mathbf{x}^l + \sigma \boldsymbol{\xi}^l \tag{4}$$

where again $\boldsymbol{\xi}^l$ is an uncorrelated, zero mean, unit variance Gaussian random variable and $\sigma \ll 1$. Importantly, noise is added to every layer, so all weights are effectively perturbed. Using $\mathcal{L}_{\boldsymbol{\xi}}(\mathbf{x}; \mathbf{w})$ to denote the loss when noise is added, the resulting learning rule is completely analogous to Eq. (3),

$$\Delta \mathbf{w}^l_{\text{np}} = -\eta \, \frac{\mathcal{L}_{\boldsymbol{\xi}}(\mathbf{x}; \mathbf{w}) - \mathcal{L}(\mathbf{x}; \mathbf{w})}{\sigma} \, \boldsymbol{\xi}^l \mathbf{x}^l \, . \tag{5}$$

Here and in what follows, two adjacent vectors indicates an outer product. So, for instance, $(\boldsymbol{\xi}^l \mathbf{x}^l)_{ij} = \xi^l_i x^l_j$.

To see that the node perturbation update, Eq. (5), provides an unbiased estimator of SGD, note first that

$$\frac{\mathcal{L}_{\boldsymbol{\xi}}(\mathbf{x}; \mathbf{w}) - \mathcal{L}(\mathbf{x}; \mathbf{w})}{\sigma} \approx \sum_{k=1}^{L} \frac{\partial \mathcal{L}(\mathbf{x}; \mathbf{w})}{\partial \mathbf{h}^k} \cdot \boldsymbol{\xi}^k \, . \tag{6}$$

Then, defining the gradient with respect to the node in layer l to be

$$\mathbf{g}^l_h \equiv \frac{\partial \mathcal{L}(\mathbf{x}; \mathbf{w})}{\partial \mathbf{h}^l} \, , \tag{7}$$

and inserting Eq. (6) into Eq. (5), we have

$$\Delta \mathbf{w}^l_{\text{np}} \approx -\eta \sum_{k=1}^{L} \mathbf{g}^k_h \cdot \boldsymbol{\xi}^k \boldsymbol{\xi}^l \mathbf{x}^l \, . \tag{8}$$

Averaging over the noise and noting that $\langle \boldsymbol{\xi}^k \boldsymbol{\xi}^l \rangle = \delta_{kl} \mathbf{I}^l$, where δ_{kl} is the Kronecker delta and $\mathbf{I}^l$ is the identity matrix for layer l, we see that

$$\langle \Delta \mathbf{w}^l_{\text{np}} \rangle \approx -\eta \, \mathbf{g}^l_h \mathbf{x}^l \, , \tag{9}$$

which is the SGD update rule in layer l. To emphasize that the node perturbation update rule is just a noisy version of SGD, we write

$$\Delta \mathbf{w}^l_{\text{np}} \approx \Delta \mathbf{w}^l_{\text{sgd}} + \eta \sum_k \mathbf{g}^k_h \cdot (\delta_{kl} \mathbf{I}^l - \boldsymbol{\xi}^k \boldsymbol{\xi}^l) \mathbf{x}^l \, . \tag{10}$$

In our simulations we use Eq. (5), but in much of our analysis we use Eq. (8), which is exact in the limit $\sigma \to 0$.

2.2 The effect of noise on performance

As can be seen explicitly in Eq. (10), node perturbation adds additional noise on top of the stochasticity inherent in SGD. To see how much noise, we can compute the covariance of the right hand side in Eq. (10),

$$\mathrm{Covar}[\Delta w_{ij}^{l}, \Delta w_{i'j'}^{l'}] = \eta^2 \left[\delta_i^l \delta_{i'}^{l'} + \delta_{ii'} \delta_{ll'} \sum_k \mathbf{g}_h^k \cdot \mathbf{g}_h^k \right] x_j^l x_{j'}^{l'} \,. \tag{11}$$

The term $\sum_k \mathbf{g}_h^k \cdot \mathbf{g}_h^k$ is large, and in fact scales with the number of units in the network. But how does that affect performance?

The short answer is that if the loss function is sufficiently convex and the noise is sufficiently large, the average loss can increase with each (noisy) gradient step. This is illustrated in Fig. 1 in two dimensions. The red contours show the loss, with equal change in loss between lines. We're taking the loss to be convex, so it's smallest at the center and increases toward the edges. The block dot shows the weight at time t, denoted $\mathbf{w}_t$, and the blue region shows, schematically, the range of possible weights after one gradient update, with the green dot indicating the average weight at time $t+1$. As can be seen by the position of the green dot, in the absence of noise the loss decreases. With the level of noise shown here, however, the loss increases on average. That's because of the convexity: weight changes in the direction opposite the gradient have a much larger effect on the loss (an increase of two units) than weight changes in the direction of the gradient (a decrease of less than one). Note that convexity is critical: if the cost function changed linearly with the weight, the average loss would equal the loss at the green point, and concavity leads to a larger decrease in average loss than would be expected from the green point.

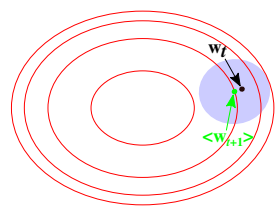

Figure 1: Effect of noise on the loss. Red lines: contour plots of the loss in two-dimensional weight space, with the loss increasing by one unit with each line; black dot: $\mathbf{w}_t$; blue shaded region: $\mathbf{w}_{t+1}$ for different realizations of the noise; green dot: $\langle \mathbf{w}_{t+1} \rangle$, where the average is over the noise. Although the noise is isotropic, its effect on the loss is not: because of the convexity, the upper limit of the noise increases the loss by two units while the lower limit decreases it by less than one. Thus, large noise coupled with convexity can cause the loss to increase even though the average gradient is in the right direction.

To quantify this, we consider the very general update rule

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_w + \eta \boldsymbol{\xi} \tag{12}$$

where $\mathbf{g}_w \equiv \langle \partial \mathcal{L} / \partial \mathbf{w} \rangle$ is the true gradient of the loss with respect to the weight and $\boldsymbol{\xi}$ is zero mean noise with covariance $\mathbf{C}$. Inserting this into the loss, Taylor expanding to second order, and averaging over the noise, the average change in loss is given approximately by

$$\langle \Delta \mathcal{L}(\mathbf{w}) \rangle \approx -2\eta |\mathbf{g}_w|^2 + \frac{\eta^2}{2} \left[\mathbf{g}_w \cdot \mathbf{H} \cdot \mathbf{g}_w + \mathrm{tr}\left[\mathbf{C} \cdot \mathbf{H} \right] \right] \tag{13}$$

where $\mathbf{H} \equiv \langle \partial^2 \mathcal{L} / \partial \mathbf{w}^2 \rangle$ is the noise-averaged Hessian. If either the covariance matrix of the noise or the convexity increases during learning, so does the loss. Any increases can, of course, always be mitigated by decreasing the learning rate. This implies, though, that the larger the noise the smaller the learning rate, and the longer it takes to learn.

Based on this analysis, we expect the following behavior: since node perturbation provides an unbiased estimate of SGD, when the learning rate is small enough we expect node perturbation and SGD to exhibit virtually identical performance. However, as the learning rate increases, performance of node perturbation should, eventually, degrade relative to SGD. That's essentially what we see, although we find that the drop in performance can be very large: at learning rates for which SGD works well, performance of node perturbation can crash to near chance.

3 Empirical Study of Node Perturbation

Does the brain use node perturbation to learn complex tasks, such as image classification? The theory discussed above suggests that node perturbation is slower than SGD, but we do not know how much slower for real world data and tasks. Given its conceptual and architectural simplicity, if node perturbation is only marginally slower, the brain may employ it to drive learning for a wide range of tasks. However, since there are many biologically plausible learning rules that achieve comparable learning efficiency with SGD in image classification tasks [18, 19, 39], if node perturbation is significantly slower than SGD, there is little reason for the brain to use it for those tasks.

We sought to understand how training time for node perturbation scales relative to SGD while varying network architecture across width, depth, and convolutions, as well as across dataset complexity (on MNIST, fashion-MNIST, and CIFAR10). While unsupervised and reinforcement learning algorithms are undoubtedly employed by the brain, supervised learning offers clear benchmarks for examining the performance of an algorithm. In addition, most past studies examining the speed of learning in the context of biologically motivated algorithms have worked in the supervised setting [19, 40, 39].

3.1 Implementation details

We developed a straightforward method to robustly and efficiently implement node perturbation via the auto-differentiation functionality of modern deep learning frameworks such as PyTorch, TensorFlow or JAX (details can be found in the next section). This approach allowed us to run carefully controlled experiments with structured architectures such as convolutional networks. We used the JAX [41] auto-differentiation neural networks library, with a single codebase and training pipeline for all experiments[1], ensuring that we matched all of the relevant choices between SGD and node perturbation (e.g., precisely the same forward pass, loss function, hyper-parameters, training loop, evaluation protocols, etc). For all design decisions, we sought to make node perturbation as fast and effective as possible, so as to establish an upper bound on its performance. In particular, we used the fast variant of node perturbation described above that uses a second forward passes through the network to establish a noiseless baseline. This variant makes circuitry requirements that the brain is unlikely to be able to fulfill. There are variants which use a noisy baseline and place fewer demands on circuitry (for example [42, 43]), but these are known to be slower. See Appendix 6.1 for details.

3.1.1 Leveraging automatic differentiation for the node perturbation update

The naive way to implement the node perturbation update is to make two separate forward passes, one with noise and one without, get the difference in the loss, and then manually calculate the weight updates for each layer based on Eq. (5). However, manual implementations of this process are frequently slow and error prone, especially for more complex architectures where parameters are shared. In the case of convolutional networks, each node in the network must be perturbed to compute gradient estimates for its weights, and then these must be combined with all of the other neurons in the same channel to compute an update for the kernel. We can, though, leverage auto-differentiation libraries to straightforwardly apply node perturbation to train network weights. This is achieved by noting that such networks still have the functional form $\mathbf{h}^l = \mathbf{w}^l(\boldsymbol{\theta}^l) \cdot \mathbf{x}^l$, where now $\mathbf{w}^l$ is a function of the underlying kernel parameters $\boldsymbol{\theta}^l$. Node perturbation gives us an estimate of the loss gradient at $\mathbf{h}^l$, $\tilde{\mathbf{g}}_h^l = (\mathcal{L}_{\boldsymbol{\xi}} - \mathcal{L})\boldsymbol{\xi}^l/\sigma$. To use auto-differentiation to find the correct gradient estimates for our parameters $\boldsymbol{\theta}^l$, we differentiate the substitute loss $\mathcal{L}_{\boldsymbol{\theta}^l} = sg(\tilde{\mathbf{g}}_h^l) \cdot \mathbf{h}^l$, where sg denotes a "stop-gradient" for the auto-differentiation, preventing any undesired backprop of the loss,

$$\frac{\partial \mathcal{L}_{\boldsymbol{\theta}^l}}{\partial \boldsymbol{\theta}^l} = \tilde{\mathbf{g}}_h^l \cdot \frac{\partial \mathbf{h}^l}{\partial \boldsymbol{\theta}^l} = \tilde{\mathbf{g}}_h^l \cdot \left(\frac{\partial \mathbf{w}^l}{\partial \boldsymbol{\theta}^l} \cdot \mathbf{x} \right) . \tag{14}$$

It can be seen that for a fully connected layer, for which $w_{ij}^l = \theta_{ij}^l$, this expression reduces to Eq. (5). This approach is general in that it can be used with any network architecture supported by the autodiff library. It is also fast in that it makes use of optimized libraries to take the gradients of the nodes with respect to weights.

[1] Code available at: `https://anonymous.4open.science/r/nodepert/`

3.2 Empirical results agree with theory

Using the ability to run like-for-like comparisons, we first compare node perturbation and SGD in a feedforward network trained on MNIST. Fig. 2 (left) shows the test set accuracy for MNIST classification during training for node perturbation (blue) and SGD (orange). In agreement with theory, for small learning rates (light blue and light orange), node perturbation and SGD track one another closely. At higher learning rates node perturbation becomes unstable: performance crashes and often fails to recover. (Crashes, which seem to be a common feature of node perturbation but not SGD, are discussed in Sec. 3.4.) To help appreciate the difference between the two algorithms, Fig. 2 (right) plots the change in the loss as a function of the learning rate step size for node perturbation and SGD. We see that for very small learning rates, the change in loss is nearly the same for both node perturbation and SGD. As the learning rate is increased, the change in loss slows and becomes positive more quickly for node perturbation. Fig. 2 (right) aligns with theory and holds for multiple snapshots (see Appendix 6.3).

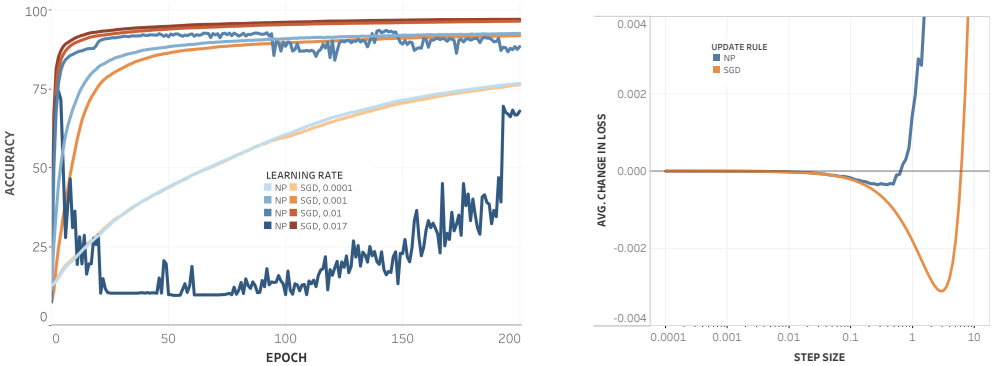

Figure 2: Left: Learning trajectories for node perturbation (blue) and SGD (orange). We trained a 2-hidden layer, fully connected network with 500 neurons in each layer on MNIST using 50 different learning rates; For clarity, we show 4 of the learning trajectories. Right: We train the same network with SGD to 93% accuracy. Then, fixing the network weights at this point, we plot the change in the MSE loss as a function of the learning rate (step size).

3.3 Scaling with network width and depth

We then investigated the scaling of node perturbation performance with network architecture. A key question for this is the choice of evaluation metric. A natural one might be the time it takes to reach a given level of performance. However, we found that for node perturbation, optimal learning rates can become very low, making learning times extremely long – often thousands of epochs. In addition, we found that the number of epochs was a very sensitive function of the target performance. Thus, instead, we trained networks for a fixed amount time (200 epochs) with learning rates uniformly sampled from the log interval $\{10^{-4}, 1\}$, and measure the final accuracy (the average test accuracy over the last 5 epochs).

Fig. 3 shows the final accuracy of node perturbation and SGD on MNIST as we increase the number of hidden units (network width; left panel) and the number of hidden layers (network depth; right panel) in the fully connected network (further plots in Appendix, Sec. 6.2). Node perturbation reaches a significantly lower final accuracy than SGD on this time scale. Moreover, increasing network depth negatively affects node perturbation much more than the width, as it produces crashes at progressively smaller learning rates (we explore the phenomenon of 'crashes' further in Sec. 3.4).

3.3.1 Assessing scalability for more challenging tasks

We found that after a few thousand epochs, node perturbation can sometimes reach the same asymptotic performance as SGD on MNIST for small MLP networks. However, this is not the case for more complex architectures and problems. To develop a clearer picture of the difference between node perturbation and SGD, we trained MLP and convolutional networks using node perturbation and SGD on MNIST, f-MNIST and CIFAR10. For these problems, we trained the networks for

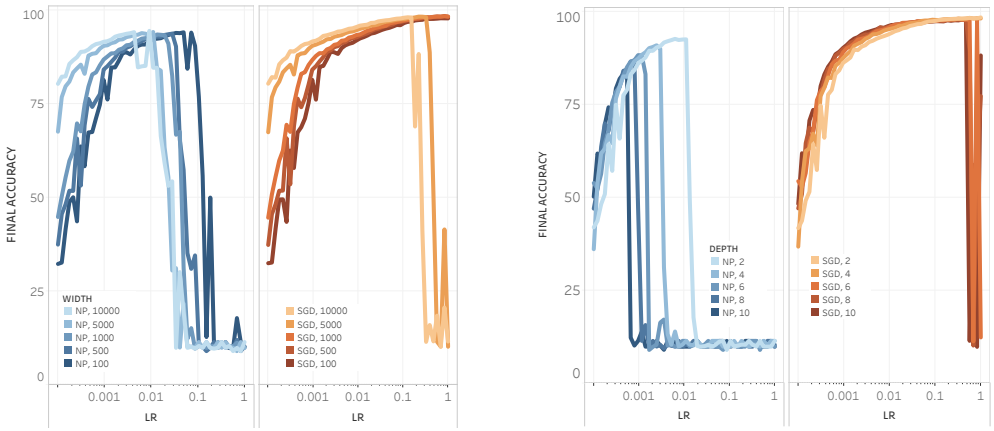

Figure 3: Final accuracy (at 200 epochs) as a function of learning rate for node perturbation and SGD while varying the width (left) and depth (right) of a fully connected network trained on MNIST.

longer, out to 5000 epochs. We evaluated the default, fully connected network (2 hidden layers, 500 neurons), a small convolutional architecture (3 hidden layers, with 64 channels each) and a large, simple convolutional network (All-CNN network [44] with ≈200,000 neurons). Table 1 gives the best achieved performances of each network after 50 and 5000 epochs and several training curves for the All-CNN architecture are shown in Fig. 4 (left panel).

To compare these networks, we optimized performance with respect to learning rate for both node perturbation and SGD. We then plotted that performance, relative to SGD at 5000 epochs, versus epoch (Fig. 4 right panel). As can be seen, the gap in performance between node perturbation and SGD grows for more complex architectures and more challenging domains, with orders of magnitude more training epochs required to reach an equivalent level of performance.

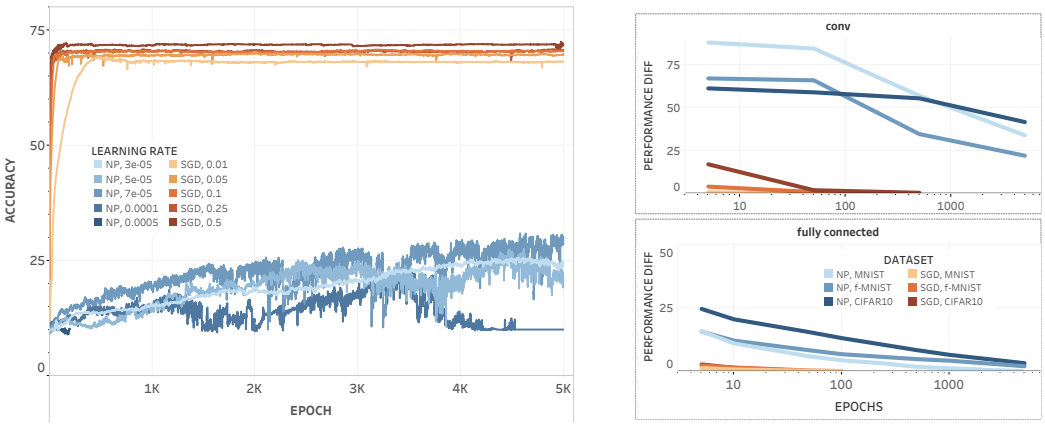

Figure 4: Left: Node perturbation (blue) and SGD (orange) training curves for the larger convolutional network trained on CIFAR10. Right: Difference between the highest accuracy achieved by the network when trained for a given number of epochs and the all-time highest accuracy of the network (trained with SGD for 5000 epochs). We stopped training when the difference vanished.

3.4 Crashes

As discussed in Sec. 2.2, whether the loss decreases during learning depends on the size of the noise relative to the true gradient. This is made explicit in Eq. (13), which shows that there are two contributions to the loss: a decrease, that's linear in the learning rate (first term on the right hand side), and a noise-induced increase, that's quadratic (second term). The second term is larger for node perturbation than for SGD, suggesting that optimal learning rates will be lower – something

Table 1: Test accuracy (%) achieved by SGD and NP for fully-connected (FC), small conv and the larger All-CNN-A conv networks on MNIST, f-MNIST and CIFAR10 when trained for a varying number of epochs.

	fully-connected				**small-conv**				**conv**			
Training Epochs	50		5000		50		5000		50		5000	
Dataset	NP	SGD	NP	SGD	NP	SGD	NP	SGD	NP	SGD	NP	SGD
MNIST	92.5	98.2	98.3	98.3	89	98.6	98.5	98.8	28	99.2	65.3	99.3
f-MNIST	81.6	89.6	87.9	89.8	74.1	91	86.7	91.4	24.4	91.2	70	92
CIFAR10	38.7	45.9	50.4	52.3	33.9	61.6	48.2	62.6	13.6	70.6	30.9	72.5

we do indeed see. However, the simple analysis that led to Eq. (13) can't tell us exactly how the second term behaves as learning progresses. Our simulations tell us that for node perturbation, it often becomes suddenly (on the scale of a several minibatch updates) large – large enough so that performance drops to near chance levels. This raises two questions. First, do we see the same thing for SGD? And second, why does the second term become large suddenly? We address these now.

To investigate the relationship between time to crash and learning rate, for both node perturbation and SGD, we used a simple 3 hidden layer network trained on MNIST. We trained for 500 epochs with 50 learning rates sampled uniformly from the log interval $\{10^{-3}, 10^{-1}\}$. The results for node perturbation are shown in the bottom right panel of Fig.5. For learning rates above 0.02, the network never learned; between 0.004 and 0.02 crashes (defined to be a drop in accuracy of at least 25%) were observed, ranging from a few epochs at high learning rate to a few hundred at low learning rate; and below 0.004 crashes were not observed out to 500 epochs. The story for SGD, however, is very different. As can be seen from the top right panel of Fig.5, at high learning rates there's no learning; at low learning rates there are no crashes. Thus, although with fine tuning it may be possible to induce crashes during SGD training, we could not find them in our simulations.

To shed light on the phenomenon of crashes, we examined the training parameters at each update step during a crash. Several of these parameters are shown in the right panel of Fig. 5 (for other parameters, see Appendix, Sec. 6.5). The main effect is an increase in weight, the magnitude of the increase is small (purple line), but the changes in the weight per time step, $|\Delta \mathbf{w}|$, increase by an order of magnitude. We speculate that the gradients are exploding, pushing the weights to a region where performance is bad and learning in difficult. However, considerable work will need to be done to determine exactly why node perturbation crashes while SGD doesn't.

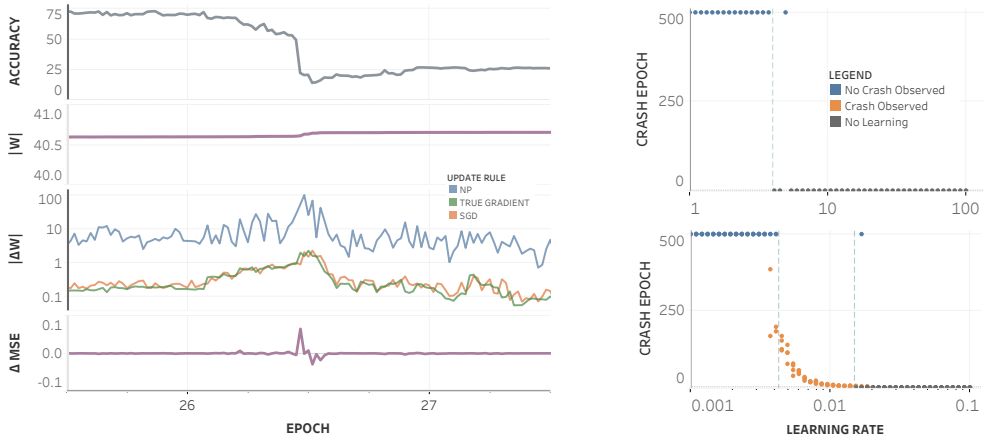

Figure 5: Left: Dynamics of accuracy, parameter norm, parameter gradient norm, and change in MSE loss during a crash. Right: mean time to crash as a function of learning rate. Orange dots shows a crash, blue dots indicate no crash and black ones indicate no learning.

4 Related Work

There is a long history in the computational literature suggesting that the brain may learn via perturbation-based mechanisms [36, 31, 23, 35, 45, 46]. This work has been motivated, no doubt, by the simplicity of perturbation-based learning mechanisms over those that employ backprop. In

particular, methods like node perturbation can operate using simple feedback delivery circuitry: global communication of a scalar performance signal is enough. This might be achieved in the brain by, for example, widespread dopaminergic connections. Perhaps the clearest didactic discussion of these ideas can be found in [23], which develops a spiking network that could learn simple tasks by reinforcing stochastic vesicle release — a form of weight perturbation.

Like us, [23] also spelled out the potential problem in larger networks: perturbation methods can be slow to learn. Just how slow they are has remained unclear, however. A few studies have examined the theory of perturbation methods [47, 26, 24]. [26] proved that node perturbation learning in shallow networks is slower than SGD by a factor that increases with the size of the network. They also empirically examined learning for a toy task in small multi-layer networks with 10 units in the input, hidden and output layers. Payeur et al. [37] examined node perturbation in a spiking network, with results suggesting that the algorithm struggles to learn CIFAR10. However, we know of no work which has attempted exhaustive optimization of node perturbation for a range of networks and tasks, making it difficult to develop a picture of whether node perturbation might be a candidate algorithm for biological learning.

Recent work has explored novel perturbation algorithms that combine perturbations with approximations of backprop which require more complex circuitry [17]. Lansdell and colleagues [48] trained the feedback pathway of a network with node perturbation, instead of the feedforward pathway. Their algorithm, which combines direct feedback alignment [18, 40] and node perturbation, achieved performance that's competitive with SGD on CIFAR10. While this mechanism sacrifices the simplicity of node perturbation, their result indicates that when node perturbation is combined with other learning mechanisms, it might play role in learning in the brain. Finally, [49, 50] present a fast and effective algorithm called Q-AGREL, which blends detailed attentional feedback with global trial-and-error signals for learning.

5 Discussion

Our experiments suggest that the performance of node perturbation relative to SGD varies significantly as a function of the network and dataset. In the case of small, shallow networks trained to perform MNIST, node perturbation can achieve levels of accuracy within $\approx$5% of SGD after 50 epochs of training. On the other hand, for large networks trained to perform CIFAR10, node perturbation remains worse than SGD by a margin of >40% after 5000 epochs of training. This appears to be part of a broader trend: the relative speed of node perturbation versus SGD drops as a function of increasing network size and task difficulty (see Table 1); for large networks and complex problems node perturbation is hundreds of times slower than SGD. We cannot draw direct conclusions about node perturbation as a learning algorithm in the brain based on these results alone. Indeed, there is neurophysiological evidence that *some* form of perturbation-based learning *is* employed by the brain [33, 34, 35]. Rather, our results suggest that node perturbation is unlikely to be fast enough to enable effective learning of some complex goal-oriented tasks that animals solve.

We focused on a simple perturbation method, node perturbation, which has been used in previous studies [26, 18, 37]. More sophisticated perturbation methods could, though, play a role in faster algorithms. Indeed, many recently developed and effective deep reinforcement learning algorithms are also perturbation-based (e.g. [51, 52]). However, these algorithms perturb only the output units of the network and use backprop with all of its associated circuitry to compute updates for the neurons and weights inside the network. These algorithms are suggestive that perturbations might be combined with more complex credit assignment motifs to achieve both fast learning and simplicity of the requisite circuitry.

5.1 Broader Impact

Researchers have suggested that artificial neural networks could serve as a framework for understanding the brain [53]. In this work, we performed experiments to understand the scalability of perturbation-based learning methods for artificial neural networks. Our results shed light on the limitations of these methods, enabling more precise questions to be framed about how the brain performs credit assignment. Beyond spurring further research, we do not anticipate any broader impacts in the short term. Ultimately, our work may help researchers uncover the class of algorithms that drive learning in the different areas of the brain. Understanding how the brain learns will have implications for technologies that directly or indirectly interface with the brain, such as brain machine interfaces, neurological rehabilitation, and drug discovery.

References

[1] Richard E Brown and Peter M Milner. The legacy of donald o. hebb: more than the hebb synapse. *Nature Reviews Neuroscience*, 4(12):1013–1019, 2003.

[2] Wulfram Gerstner, Richard Kempter, J Leo Van Hemmen, and Hermann Wagner. A neuronal learning rule for sub-millisecond temporal coding. *Nature*, 383(6595):76–78, 1996.

[3] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.

[4] Seppo Linnainmaa. Taylor expansion of the accumulated rounding error. *BIT Numerical Mathematics*, 16(2):146–160, 1976.

[5] Awni Hannun, Carl Case, Jared Casper, Bryan Catanzaro, Greg Diamos, Erich Elsen, Ryan Prenger, Sanjeev Satheesh, Shubho Sengupta, Adam Coates, et al. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014.

[6] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.

[7] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115(3):211–252, 2015.

[8] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, łLukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.

[9] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016.

[10] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.

[11] Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemysław Dębiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, et al. Dota 2 with large scale deep reinforcement learning. *arXiv preprint arXiv:1912.06680*, 2019.

[12] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.

[13] Sergey Levine, Peter Pastor, Alex Krizhevsky, Julian Ibarz, and Deirdre Quillen. Learning hand-eye coordination for robotic grasping with deep learning and large-scale data collection. *The International Journal of Robotics Research*, 37(4-5):421–436, 2018.

[14] Serkan Cabi, Sergio Gómez Colmenarejo, Alexander Novikov, Ksenia Konyushkova, Scott Reed, Rae Jeong, Konrad Zolna, Yusuf Aytar, David Budden, Mel Vecerik, Oleg Sushkov, David Barker, Jonathan Scholz, Misha Denil, Nando de Freitas, and Ziyu Wang. Scaling data-driven robotics with reward sketching and batch reinforcement learning. *arXiv preprint arXiv:1909.12200v3*, 2020.

[15] Stephen Grossberg. Competitive learning: From interactive activation to adaptive resonance. *Cognitive science*, 11(1):23–63, 1987.

[16] Francis Crick. The recent excitement about neural networks. *Nature*, 337(6203):129, 1989.

[17] Timothy P Lillicrap, Adam Santoro, Luke Marris, Colin J Akerman, and Geoffrey Hinton. Backpropagation and the brain. *Nature Reviews Neuroscience*, 21(6):335–346, 2020.

[18] Timothy P Lillicrap, Daniel Cownden, Douglas B Tweed, and Colin J Akerman. Random synaptic feedback weights support error backpropagation for deep learning. *Nature communications*, 7(1):1–10, 2016.

[19] Dong-Hyun Lee, Saizheng Zhang, Asja Fischer, and Yoshua Bengio. Difference target propagation. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 498–515. Springer, 2015.

[20] Joao Sacramento, Rui Ponte Costa, Yoshua Bengio, and Walter Senn. Dendritic error backpropagation in deep cortical microcircuits. *arXiv preprint arXiv:1801.00062*, 2017.

[21] James CR Whittington and Rafal Bogacz. An approximation of the error backpropagation algorithm in a predictive coding network with local hebbian synaptic plasticity. *Neural computation*, 29(5):1229–1262, 2017.

[22] Roman Pogodin and Peter E Latham. Kernelized information bottleneck leads to biologically plausible 3-factor hebbian learning in deep networks. *arXiv preprint arXiv:2006.07123*, 2020.

[23] H Sebastian Seung. Learning in spiking neural networks by reinforcement of stochastic synaptic transmission. *Neuron*, 40(6):1063–1073, 2003.

[24] Ronald J Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8(3-4):229–256, 1992.

[25] Bernard Widrow and Michael A Lehr. 30 years of adaptive neural networks: perceptron, madaline, and backpropagation. *Proceedings of the IEEE*, 78(9):1415–1442, 1990.

[26] Justin Werfel, Xiaohui Xie, and H Sebastian Seung. Learning curves for stochastic gradient descent in linear feedforward networks. In *Advances in neural information processing systems*, pages 1197–1204, 2004.

[27] T. Branco and K. Staras. The probability of neurotransmitter release: variability and feedback control at single synapses. *Nat. Rev. Neurosci.*, 10:373–383, 2009.

[28] Michael London, Arnd Roth, Lisa Beeren, Michael Häusser, and Peter E Latham. Sensitivity to perturbations in vivo implies high noise and suggests rate coding in cortex. *Nature*, 466(7302):123–127, 2010.

[29] Sho Yagishita, Akiko Hayashi-Takagi, Graham CR Ellis-Davies, Hidetoshi Urakubo, Shin Ishii, and Haruo Kasai. A critical time window for dopamine actions on the structural plasticity of dendritic spines. *Science*, 345(6204):1616–1620, 2014.

[30] Simon N Jacob and Hendrikje Nienborg. Monoaminergic neuromodulation of sensory processing. *Frontiers in neural circuits*, 12:51, 2018.

[31] KP Unnikrishnan and Kootala P Venugopal. Alopex: A correlation-based learning algorithm for feedforward and recurrent neural networks. *Neural Computation*, 6(3):469–490, 1994.

[32] Thomas Miconi. Biologically plausible learning in recurrent neural networks reproduces neural dynamics observed during cognitive tasks. *Elife*, 6:e20899, 2017.

[33] Ila R Fiete, Michale S Fee, and H Sebastian Seung. Model of birdsong learning based on gradient estimation by dynamic perturbation of neural conductances. *Journal of neurophysiology*, 98(4):2038–2057, 2007.

[34] Jorgen Kornfeld, Michał Januszewski, P Schubert, Viren Jain, Winfried Denk, and Michale S Fee. An anatomical substrate of credit assignment in reinforcement learning. *BioRxiv*, 2020.

[35] Guy Bouvier, Johnatan Aljadeff, Claudia Clopath, Célian Bimbard, Jonas Ranft, Antonin Blot, Jean-Pierre Nadal, Nicolas Brunel, Vincent Hakim, and Boris Barbour. Cerebellar learning using perturbations. *Elife*, 7:e31599, 2018.

[36] Pietro Mazzoni, Richard A Andersen, and Michael I Jordan. A more biologically plausible learning rule for neural networks. *Proceedings of the National Academy of Sciences*, 88(10):4433–4437, 1991.

[37] Alexandre Payeur, Jordan Guerguiev, Friedemann Zenke, Blake Richards, and Richard Naud. Burst-dependent synaptic plasticity can coordinate learning in hierarchical circuits. *bioRxiv*, 2020.

[38] Marwan Jabri and Barry Flower. Weight perturbation: An optimal architecture and learning technique for analog vlsi feedforward and recurrent multilayer networks. *IEEE Transactions on Neural Networks*, 3(1):154–157, 1992.

[39] Sergey Bartunov, Adam Santoro, Blake Richards, Luke Marris, Geoffrey E Hinton, and Timothy Lillicrap. Assessing the scalability of biologically-motivated deep learning algorithms and architectures. In *Advances in Neural Information Processing Systems*, pages 9368–9378, 2018.

[40] Arild Nøkland. Direct feedback alignment provides learning in deep neural networks. *arXiv preprint arXiv:1609.01596*, 2016.

[41] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, and Skye Wanderman-Milne. JAX: composable transformations of Python+NumPy programs, 2018.

[42] Tatsuya Cho, Kentaro Katahira, Kazuo Okanoya, and Masato Okada. Node perturbation learning without noiseless baseline. *Neural networks*, 24(3):267–272, 2011.

[43] Kazuyuki Hara, Kentaro Katahira, and Masato Okada. Statistical mechanics of node-perturbation learning with noisy baseline. *Journal of the Physical Society of Japan*, 86(2):024002, 2017.

[44] Jost Tobias Springenberg, Alexey Dosovitskiy, Thomas Brox, and Martin Riedmiller. Striving for simplicity: The all convolutional net. *arXiv preprint arXiv:1412.6806*, 2014.

[45] Victor Pedrosa and Claudia Clopath. The role of neuromodulators in cortical plasticity. a computational perspective. *Frontiers in synaptic neuroscience*, 8:38, 2017.

[46] Johnatan Aljadeff, James D'amour, Rachel E Field, Robert C Froemke, and Claudia Clopath. Cortical credit assignment by hebbian, neuromodulatory and inhibitory plasticity. *arXiv preprint arXiv:1911.00307*, 2019.

[47] James C Spall. Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. *IEEE transactions on automatic control*, 37(3):332–341, 1992.

[48] Benjamin James Lansdell, Prashanth Ravi Prakash, and Konrad Paul Kording. Learning to solve the credit assignment problem. In *International Conference on Learning Representations*, 2020.

[49] Pieter R Roelfsema and Arjen van Ooyen. Attention-gated reinforcement learning of internal representations for classification. *Neural computation*, 17(10):2176–2214, 2005.

[50] Isabella Pozzi, Sander Bohté, and Pieter Roelfsema. A biologically plausible learning rule for deep learning in the brain. *arXiv preprint arXiv:1811.01768*, 2018.

[51] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PMLR, 2016.

[52] John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *International conference on machine learning*, pages 1889–1897. PMLR, 2015.

[53] Blake A Richards, Timothy P Lillicrap, Philippe Beaudoin, Yoshua Bengio, Rafal Bogacz, Amelia Christensen, Claudia Clopath, Rui Ponte Costa, Archy de Berker, Surya Ganguli, et al. A deep learning framework for neuroscience. *Nature neuroscience*, 22(11):1761–1770, 2019.

[54] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of machine learning research*, 13(2), 2012.

[55] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

Checklist

1. For all authors...
   (a) Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope? [Yes]
   (b) Did you describe the limitations of your work? [Yes]
   (c) Did you discuss any potential negative societal impacts of your work? [Yes] , see Section 5.1
   (d) Have you read the ethics review guidelines and ensured that your paper conforms to them? [Yes]
2. If you are including theoretical results...
   (a) Did you state the full set of assumptions of all theoretical results? [Yes]
   (b) Did you include complete proofs of all theoretical results? [Yes]
3. If you ran experiments...
   (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [Yes]
   (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [Yes]
   (c) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [Yes]
   (d) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [Yes]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
   (a) If your work uses existing assets, did you cite the creators? [Yes]
   (b) Did you mention the license of the assets? [N/A]
   (c) Did you include any new assets either in the supplemental material or as a URL? [No]
   (d) Did you discuss whether and how consent was obtained from people whose data you're using/curating? [N/A]
   (e) Did you discuss whether the data you are using/curating contains personally identifiable information or offensive content? [N/A]
5. If you used crowdsourcing or conducted research with human subjects...
   (a) Did you include the full text of instructions given to participants and screenshots, if applicable? [N/A]
   (b) Did you describe any potential participant risks, with links to Institutional Review Board (IRB) approvals, if applicable? [N/A]
   (c) Did you include the estimated hourly wage paid to participants and the total amount spent on participant compensation? [N/A]

6 Appendix

6.1 Further Implementation Details

The default training setup for our experiments was the following: loss function (*Mean Squared Error*), injected noise (*Gaussian iid, mean=0, variance*=10^{-6}), training a subset of layers versus all layers simultaneously (*All*), weight decay (*None*) and dataset preprocessing (*Data Standardization*). We exhaustively varied each of these hyper-parameters to find values that worked well for node perturbation on the MNIST dataset and then fixed these for all experiments. Some of the results were surprising to us: for example, the MSE loss was found to produce better results with node perturbation than the standard cross-entropy loss. We used mini-batches of size *100* for all experiments and used the same parameters for SGD. Further experimental details may be found in the Appendix. As expected, *learning rate* was found to be a dominant parameter for determining learning speed of the algorithms. We fixed all other hyperparameters and optimized the learning rate for each new network architecture and dataset via random search [54].

The experiments reported in the paper were implemented in Python and JAX. We open source the code on our git repository and encourage interested researchers to use our code base to design further experiments. We run the experiments on our laboratory's local cluster which consists of a combination of CPUs and GPUs (8G Nvidia GeForce RTX 2080 and 16G Nvidia P5000) with the computationally expensive, convolution experiments being run on the GPU. The most computationally expensive experiments were the longer runs (5000 epochs) on the All-CNN architecture, requiring about 2 days to finish.

6.2 Scaling with network width and depth

Similar to our setup for Fig. 3, we trained networks for a fixed amount time (200 epochs) with learning rates uniformly sampled from the log interval $\{10^{-4}, 1\}$, and measure the final accuracy (the average test accuracy over the last 5 epochs) on MNIST. Fig. 6 shows a box plot of the final accuracy of node perturbation and SGD as we increase the number of hidden units (network width; left panel) and the number of hidden layers (network depth; right panel) in the fully connected network.

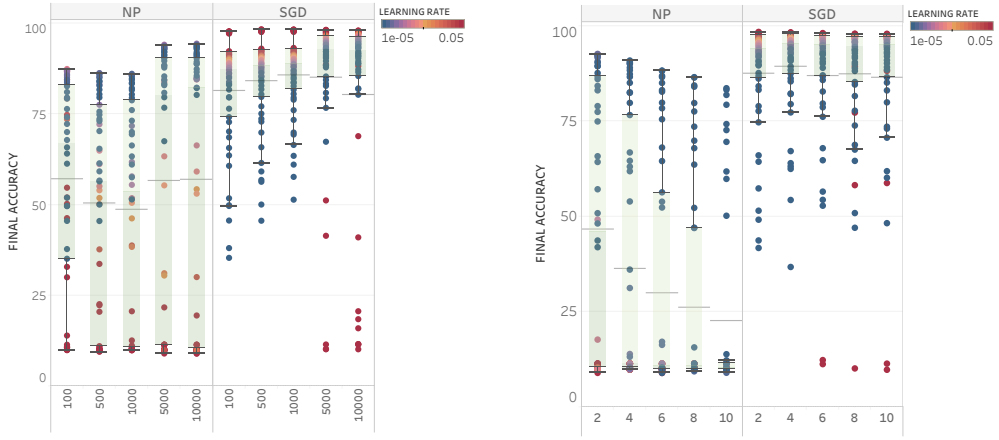

Figure 6: Plot showing the final accuracy (after 200 epochs) versus learning rate for node perturbation and SGD while varying the width (left) and depth (right) of a fully connected network when trained on MNIST.

In the setup above, we varied the number of neurons (width) of a single hidden layer neural network. Fig. 7 shows the final accuracy (the average test accuracy over the last 5 epochs) on MNIST for a 2 hidden layer network as we vary the number of neurons in the network width.

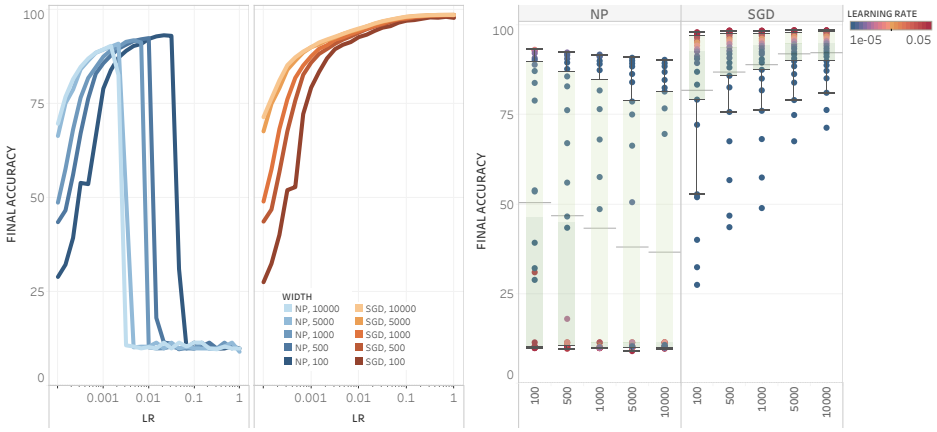

Figure 7: Plot showing the final accuracy (after 200 epochs) versus learning rate for node perturbation and SGD while varying the width of a 2 hidden layer fully connected network when trained on MNIST.

6.3 Linesearches for different network architectures

Similar to Fig.2 (right panel), we perform linesearch and compare node perturbation and SGD for a fully connected network. We train the network until 93% performance on MNIST and then freeze the weights. We sample 50 different learning rates, η, and plot the average loss, $\langle \delta L \rangle_z$ for each of these in the figure below across networks with different widths and depth. When the learning rate, η, exceeds η^*, learning reverses and performance starts to drop.

6.3.1 Network Width

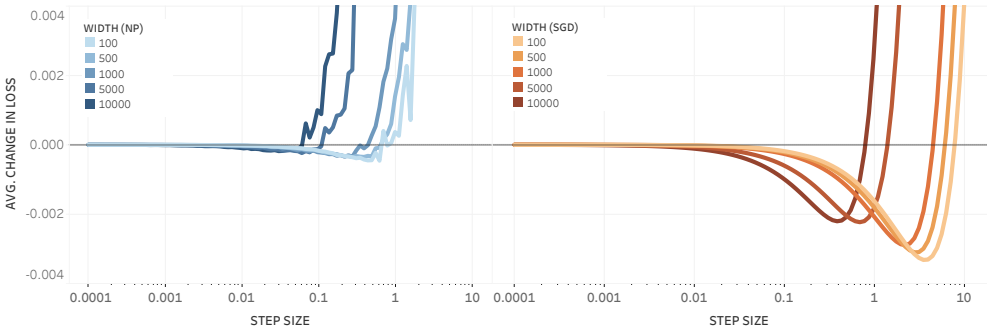

Figure 8: Figure indicating how average loss, $\langle \delta L \rangle_z$, behaves when plotted against learning rate on MNIST, η, for node perturbation and SGD with a single hidden layer, fully connected network while varying the number of neurons in the hidden layer.

6.3.2 Network Depth

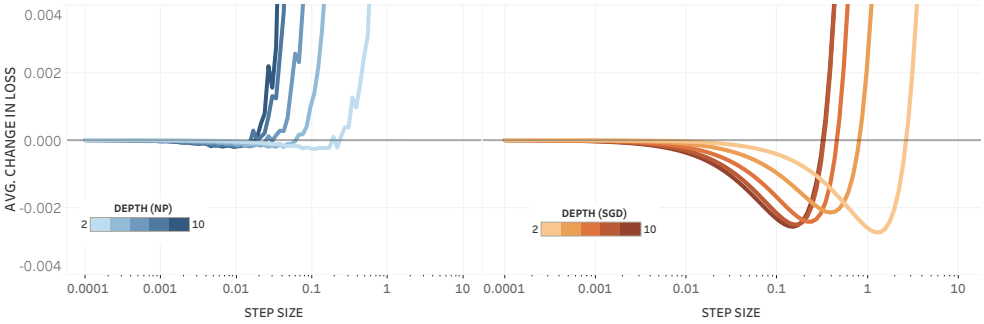

Figure 9: Figure indicating how average loss, $\langle \delta L \rangle_z$, behaves when plotted against learning rate on MNIST, η, for node perturbation and SGD with a fully connected network while varying the number of hidden layers, each with 500 neurons.

6.4 Additional Learning Trajectories

Fig. 10 (left) show a subset of the learning curves for a small convolutional network (3 layers, 32 channels in each layer, no maxpooling) trained on CIFAR10. Fig.10 (right) shows the training curves for the larger convolutional network (All-CNN) across multiple learning rates trained for a much larger number of epochs on simpler datasets like MNIST and fashion-MNIST. Even on simpler datasets, this larger network (≈200,000 neurons) is not able to get comparative performance when trained with node perturbation.

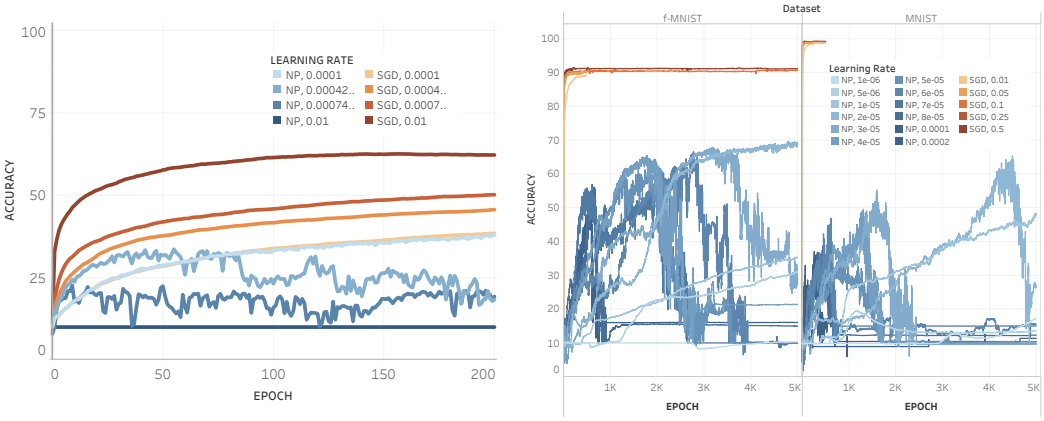

Figure 10: Training curves for a small convolutional network on CIFAR10 (left) and training curves for the larger convolutional network on MNIST and f-MNIST (right).

6.5 Further experiments on crashes

For all the experiments on crashes, we use a 3 hidden layer, fully connected network (500 neurons in each hidden layer) since crashes are observed over a much wider range of learning rates as compared to a network with 2 hidden layers. We use a batchsize of 100 and results are reported on MNIST. We defined a crash as a drop in the accuracy of 25% from the highest accuracy reached during training so far. To shed light on the phenomenon of crashes, we examined at the training parameters at each

update step during a crash such as the gradient norm, weight norm, change in loss, sign symmetry, angle with true gradient, etc (See Sec6.5.3 onwards).

6.5.1 Implementation details

As calculating these values for every update step is quite computationally expensive, we use only 10% of the MNIST training set for these evaluations. We proceed with the usual training method and store a checkpoint of the network every 5 epochs. Once a crash is observed, we restart training from this checkpoint, but this time computing all the required metrics.

6.5.2 Crash Trajectories

Shown below are three examples of learning curves illustrating the phenomenon of crashes, while using 10% of the MNIST dataset (we use a subset of the dataset so that we can calculate the exact true gradient over all the training samples).

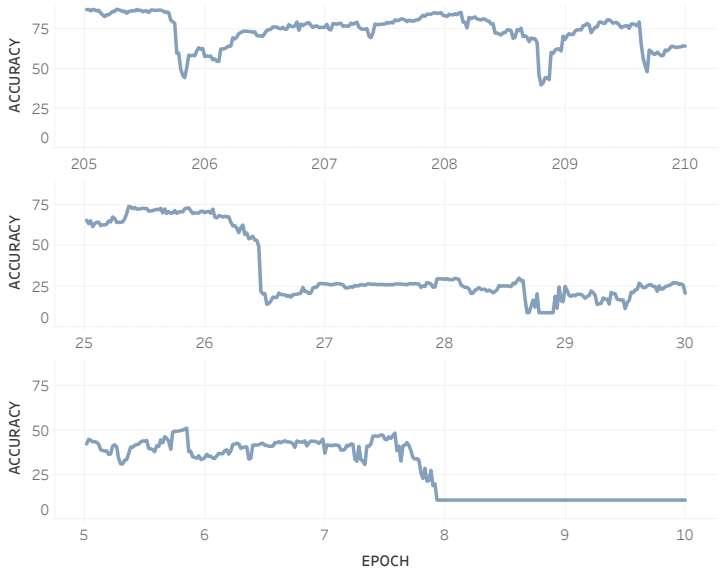

Figure 11: Different type of training trajectories when a crash is observed. Most times the network drops chance levels, but this is not always the case. Sometimes, the network recovers and at other times there is no change in the accuracy after the crash.

6.5.3 Gradient norms and gradient ratios during a crash

Here we measure the norms of the gradients and gradient ratios (approximated gradient (i.e. node perturbation or SGD) and true gradient) for each of the layers of the neural network at every update step.

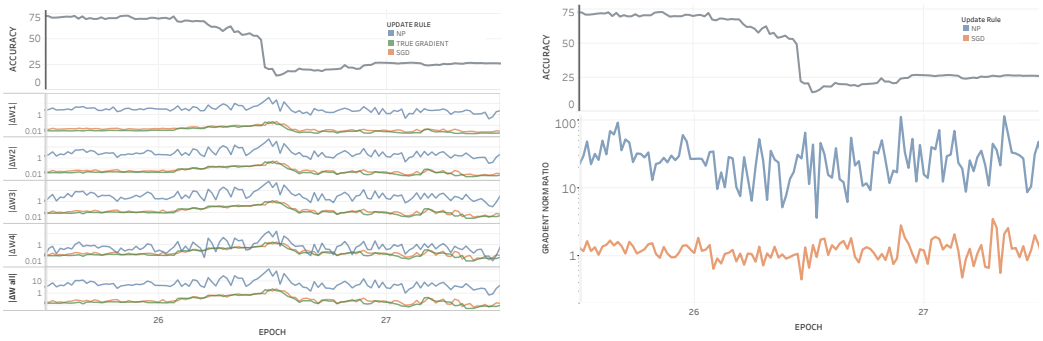

Figure 12: Norm of the gradients (left) and the ratio between the approximated gradient and the true gradient (right) for each layer of the neural network during a crash.

6.5.4 Weight norms and activity norms during a crash

Here we measure the norms of the parameters (network weights) and activities of the neurons for each of the layers of the neural network at every update step on MNIST during a crash.

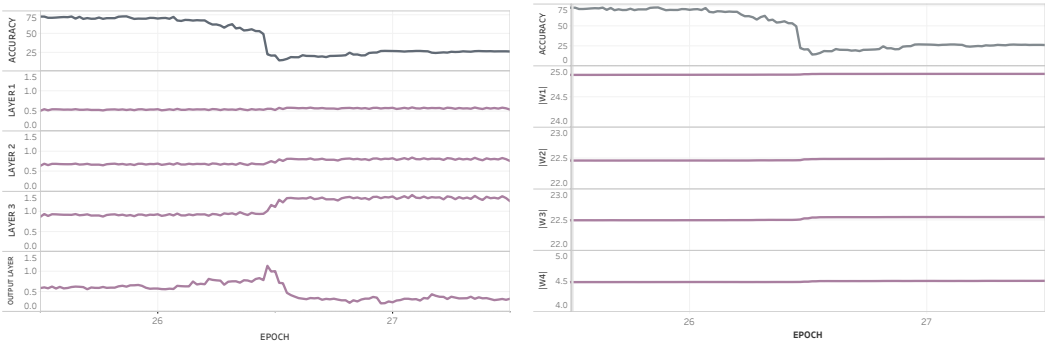

Figure 13: Norms of the weights (left panel) and activities of the neurons (right panel) for each layer of the fully connected network calculate at every minibatch update.

6.6 Additional Experiments

6.6.1 Effect of minibatch size

Here, we see the relative effect of minibatch size for node perturbation and SGD. Our setup is the default one: fully connected network with 2 hidden layers and 500 neurons in each layer. We train the network on MNIST and compare the final accuracies after 60,000 batch updates (i.e. there are 60,000 training examples in MNIST, so for a minibatch size of 1, that would mean training for 1 epoch, for minibatch size of 10, it would be 10 epochs, and so on). We uniformly sampled 50 learning rates and plot the final accuracies in Fig.14.

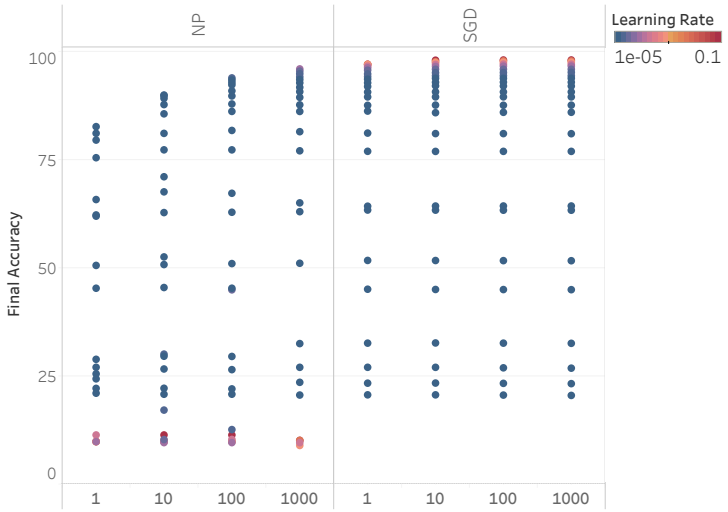

Figure 14: The final performance achieved by a 2 hidden layer fully connected network on MNIST when trained for 60,000 minibatch updates.

6.6.2 Exploring adaptive learning rates optimizer

Here, we perform preliminary experiments to get an idea of whether using an optimizer with adaptive learning rate (e.g. Adam [55]) would significantly help the performance of node perturbation? The step size is the same as the learning rate. Our setup is the default one: fully connected network with 2 hidden layers and 500 neurons in each layer. We train the network with node perturbation, node perturbation with an adam-like update, SGD and Adam for 200 epochs with 50 uniformly sampled learning rates and plot the final accuracies in Fig.15.

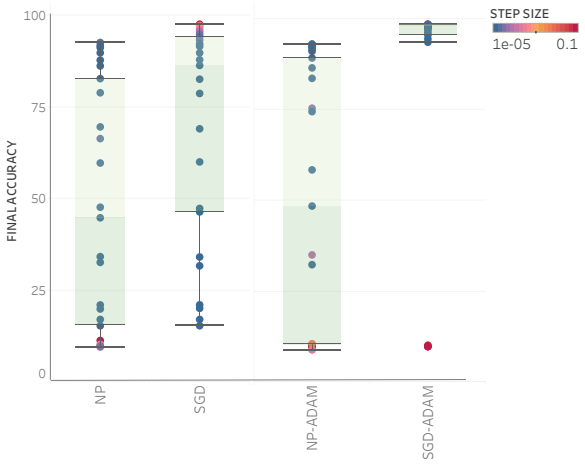

Figure 15: The final performance of introducing an adam-like update for gradients approximated by node perturbation on MNIST with the default fully connected network.